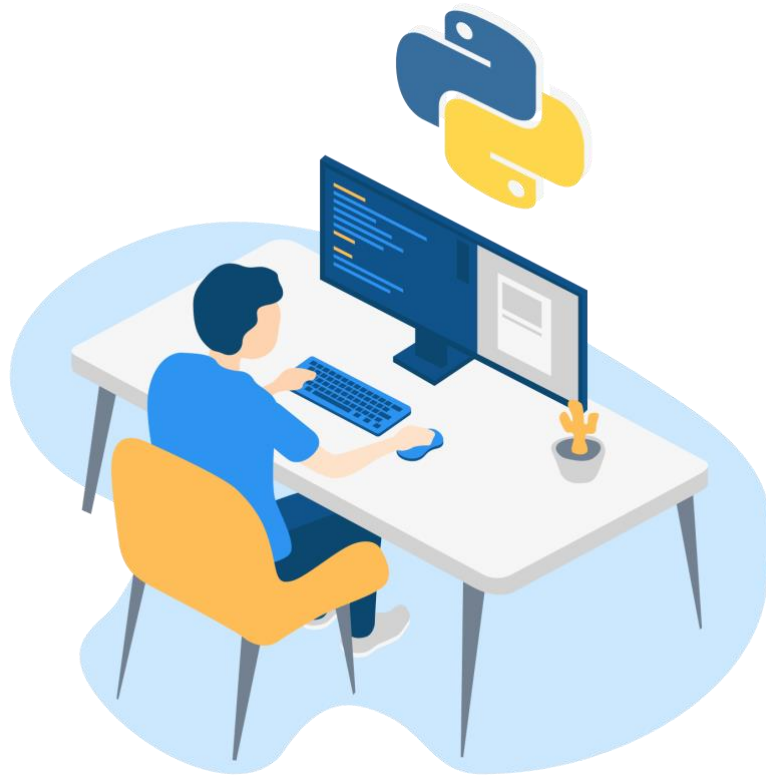




6.1 Tujuan

Setelah menyelesaikan bab ini dan mengikuti kegiatan praktikum, mahasiswa diharapkan mampu:

1. Menyiapkan lingkungan pengembangan Django yang terisolasi menggunakan *virtual environment*.
2. Membuat dan menjalankan proyek Django serta memahami struktur direktori dasarnya.
3. Mendefinisikan model data menggunakan `models.Model` di Django untuk merepresentasikan struktur basis data.
4. Menerapkan perubahan model ke dalam basis data melalui proses migrasi (*migrations*).
5. Mendaftarkan model ke Django Admin dan menggunakannya untuk mengelola data aplikasi.

6.2 Pengantar

Pada bab-bab sebelumnya, kita telah membangun halaman web statis menggunakan HTML, CSS, dan sedikit interaktivitas dengan JavaScript. Sekarang, kita akan melangkah ke dunia pengembangan web backend menggunakan Django, sebuah framework web tingkat tinggi berbasis Python yang memungkinkan pengembangan aplikasi web yang kompleks dan aman dengan cepat.

6.2.1 Apa itu Django?

Django adalah *framework* Python gratis dan *open-source* yang mengikuti pola arsitektur *Model-View-Template* (MVT). Filosofi utamanya adalah "*batteries-included*", yang berarti Django menyediakan hampir semua yang dibutuhkan pengembang secara *out-of-the-box*, mulai dari sistem autentikasi, panel admin, hingga ORM (*Object-Relational Mapper*) yang kuat.

6.2.2 Virtual Environment

Sebelum memulai proyek, sangat penting untuk menggunakan *virtual environment*. Bayangkan komputer kamu itu seperti sebuah kos-kosan yang besar. Di kos-kosan itu ada dapur umum yang bisa dipakai semua penghuni. Masalahnya, kalau semua orang simpan bahan makanan di sana, bahan makanan jadi campur aduk: ada orang yang butuh garam versi organik, ada yang butuh garam meja biasa, ada yang butuh gula aren. Dengan *virtual environment*, kamu seperti punya dapur kecil pribadi di dalam dapur umum tersebut. Kamu bisa menyimpan bahan dan alatmu sendiri, tanpa khawatir tercampur dengan orang lain.

6.2.3 Struktur Proyek Django

Ketika kita membuat proyek Django, akan dihasilkan struktur file dan direktori standar seperti berikut :

- `manage.py`: Sebuah utilitas baris perintah yang memungkinkan kita berinteraksi dengan proyek Django. Kita akan sering menggunakannya untuk menjalankan server, membuat migrasi, dan tugas-tugas lainnya.
- Direktori Proyek Utama (misalnya, `mysite/`):
 - `settings.py`: Berisi semua konfigurasi untuk proyek Django, seperti koneksi basis data, aplikasi yang terinstal, dan zona waktu.
 - `urls.py`: Tempat untuk mendefinisikan URL-URL untuk situs. `Urls.py` adalah "daftar isi" dari web.
 - `wsgi.py`: Titik masuk untuk server web yang kompatibel dengan WSGI untuk melayani proyek.

6.2.4 Konsep Model dan ORM

Dalam Django, Model adalah representasi struktur data aplikasi dalam bentuk class Python. Setiap class Model memetakan ke satu tabel di basis data. Atribut di dalam kelas tersebut memetakan ke kolom-kolom tabel.

Django dilengkapi dengan *Object-Relational Mapper* (ORM), yang merupakan jembatan antara kode Python yang ditulis (objek) dan basis data relasional (tabel). Dengan ORM, Kita tidak perlu menulis query SQL secara manual. Kita cukup berinteraksi dengan objek Python, dan Django akan menerjemahkannya menjadi query SQL yang sesuai secara otomatis.

```
1. # Contoh sebuah Model di file models.py
2. from django.db import models
3.
4. class Post(models.Model):
5.     author = models.ForeignKey('auth.User', on_delete=models.CASCADE)
6.     title = models.CharField(max_length=200)
7.     text = models.TextField()
8.     created_date = models.DateTimeField(default=timezone.now)
```

6.2.5 Migrasi

Migrasi adalah cara Django untuk menyebarkan perubahan yang kita buat pada model ke dalam skema basis data. Prosesnya terdiri dari dua langkah utama:

1. `makemigrations`: Django akan mendeteksi perubahan pada `models.py` dan membuat file migrasi baru yang berisi instruksi untuk memperbarui basis data.
2. `migrate`: Django akan menerapkan file-file migrasi yang tertunda ke basis data dan secara efektif mengubah struktur tabelnya agar sesuai dengan model.

6.2.6 Django Admin

Salah satu fitur paling menonjol dari Django adalah antarmuka admin yang dibuat secara otomatis. Setelah kita mendefinisikan model, kita dapat mendaftarkannya ke panel admin hanya dengan beberapa baris kode. Django kemudian akan menyediakan antarmuka web yang fungsional untuk membuat, melihat, memperbarui, dan menghapus (*CRUD*) data dalam model tersebut.

6.3 Kegiatan Praktikum

Langkah 1: Persiapan

Pastikan python sudah terinstall pada sistem, jika belum silahkan unduh dari <https://www.python.org/downloads/>. Pada saat modul ini dibuat python versi terbaru adalah 3.13.

Pastikan juga Visual Studio Code sudah terinstall dengan tambahan ekstensi python.

Langkah 2 : Persiapan Environment dan Instalasi Django

1. Buat sebuah folder baru untuk kegiatan praktikum dan beri nama `django`. Buka folder tersebut melalui visual studio code dengan cara klik `File` → `Open Folder`.
2. Pastikan posisi visual studio code sudah membuka folder `django`. Aktifkan terminal pada visual studio code dengan klik `View` → `Terminal` atau tekan `Ctrl+``. Ketik perintah berikut untuk membuat *virtual environment* baru.

Terminal

```
1. python -m venv myenv
```

3. Aktifkan *virtual environment* tersebut.

Terminal

```
1. myenv\Scripts\activate.bat
```

4. Buat file baru pada folde `django`, beri nama `requirements.txt` dan isi dengan kode berikut

```
1. Django~=5.1.2
```

5. Instal Django menggunakan `pip`.

Terminal

```
1. pip install -r requirements.txt
```

Langkah 3: Membuat Proyek Django

1. Gunakan perintah `django-admin` untuk membuat proyek baru.

Terminal

```
1. django-admin startproject mysite .
```

Perhatikan tanda titik (.) di akhir, yang berarti proyek dibuat di direktori saat ini.

2. Buka file `mysite/settings.py`. Ubah zona waktu agar sesuai dengan lokasi kita.

`mysite/settings.py`

```
1. # Internationalization
2. # https://docs.djangoproject.com/en/5.2/topics/i18n/
3.
4. LANGUAGE_CODE = 'en-us'
```

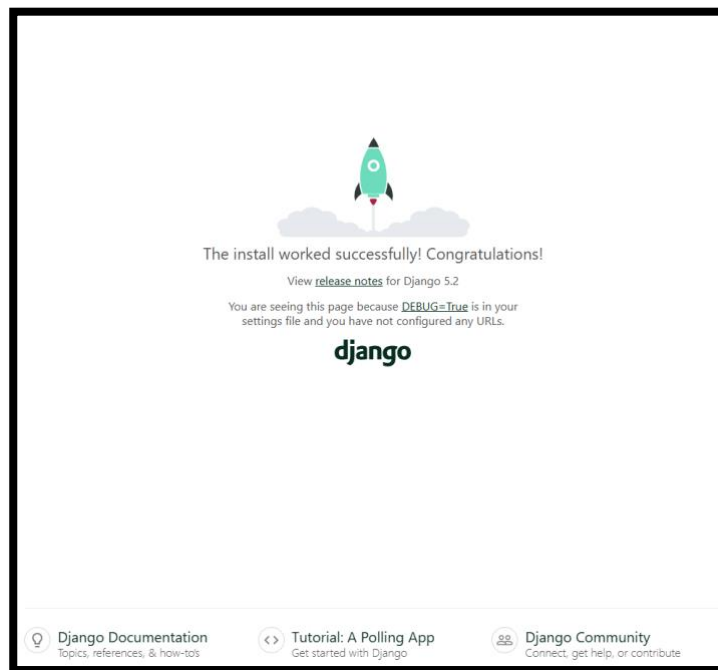
```
5. TIME_ZONE = 'Asia/Jakarta'
6. USE_I18N = True
7. USE_TZ = True
```

3. Jalankan server django untuk pertama kalinya.

Terminal

```
1. python manage.py runserver
```

4. Buka browser dan kunjungi <http://127.0.0.1:8000/>. Untuk menghentikan server tekan Ctrl + C pada terminal Visual Studio Code



Langkah 4: Membuat Aplikasi Blog

1. Dalam Django, sebuah proyek terdiri dari beberapa "aplikasi". Mari buat aplikasi untuk blog kita.

Terminal

```
1. python manage.py startapp blog
```

2. Daftarkan aplikasi blog yang baru ke dalam proyek. Buka `mysite/settings.py` dan tambahkan `blog` ke daftar `INSTALLED_APPS`.

`mysite/settings.py`

```
1. # Di dalam mysite/settings.py
2. INSTALLED_APPS = [
3.     'django.contrib.admin',
4.     'django.contrib.auth',
5.     'django.contrib.contenttypes',
```

```
6.     'django.contrib.sessions',
7.     'django.contrib.messages',
8.     'django.contrib.staticfiles',
9.     'blog', # Tambahkan baris ini
10.]
```

Langkah 5: Membuat Model Post

1. Buka file `blog/models.py`. Hapus konten default dan definisikan model untuk postingan blog.

`blog/models.py`

```
1. from django.conf import settings
2. from django.db import models
3. from django.utils import timezone
4.
5. class Post(models.Model):
6.     author = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete
    =models.CASCADE)
7.     title = models.CharField(max_length=200)
8.     text = models.TextField()
9.     created_date = models.DateTimeField(default=timezone.now)
10.    published_date = models.DateTimeField(blank=True, null=True)
11.
12.    def publish(self):
13.        self.published_date = timezone.now()
14.        self.save()
15.
16.    def __str__(self):
17.        return self.title
```

Model `Post` ini berisi atribut:

- Penulis (`author`)
- Judul (`title`)
- Isi (`text`)
- Tanggal dibuat (`created_date`)
- Tanggal dipublish (`published_date`)

Selain itu ada dua method:

- `publish()` → untuk mempublish post.
- `__str__()` → untuk menampilkan judul ketika objek dipanggil sebagai string.

Langkah 6: Membuat dan Menjalankan Migrasi

1. Setelah model didefinisikan, gunakan perintah ini untuk membuat file migrasi.

Terminal

```
1. python manage.py makemigrations blog
```

2. Sekarang, terapkan migrasi tersebut ke basis data.

Terminal

```
1. python manage.py migrate
```

Basis data default Django adalah file SQLite yang akan dibuat otomatis (db.sqlite3).

Langkah 7: Mendaftarkan Model ke Panel Admin

1. Agar kita bisa mengelola `Post` melalui panel admin, kita perlu mendaftarkannya. Buka file `blog/admin.py`. tambahkan kode berikut

`blog/admin.py`

```
1. from django.contrib import admin
2. from .models import Post
3.
4. admin.site.register(Post)
```

2. Buat akun superuser untuk bisa login ke panel admin.

Terminal

```
1. python manage.py createsuperuser
```

Ikuti prompt untuk membuat username, email, dan password.

Username : admin

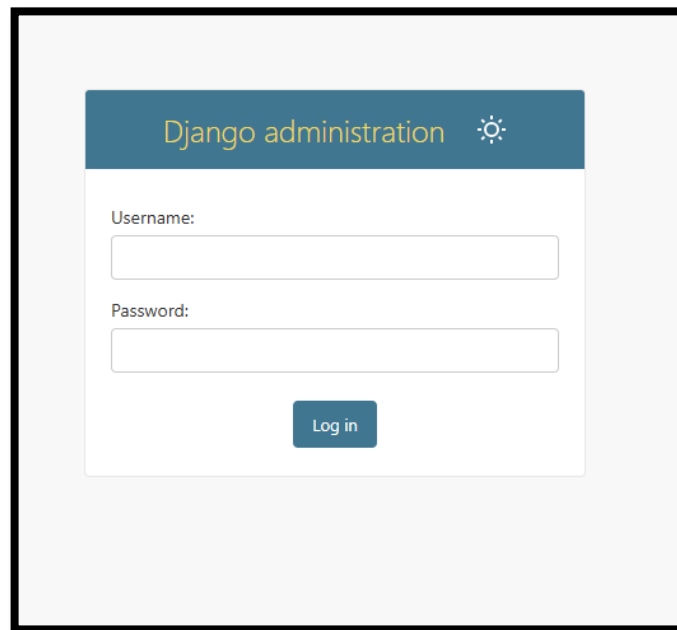
Email : admin@admin.com

Password : 4dm1nptiums123

3. Jalankan kembali server dan kunjungi `http://127.0.0.1:8000/admin/`. Login dengan akun yang baru saja Anda buat

Terminal

```
1. python manage.py runserver
```



4. Klik tombol +Add pada Posts kemudian buatlah 3 isian postingan untuk blog kita. Jangan lupa klik Save setiap kali selesai membuat postingan blog. Note : kotak merah harus terisi semua

6.4 Rangkuman

- Pengembangan aplikasi Django harus selalu dimulai dengan pembuatan dan aktivasi *virtual environment* untuk mengisolasi dependensi.

- Sebuah proyek Django adalah kumpulan konfigurasi dan aplikasi. Sebuah aplikasi adalah modul yang melakukan fungsi tertentu (misalnya, blog, polling).
- Model (`models.py`) adalah jantung dari data aplikasi, didefinisikan sebagai kelas Python yang dipetakan ke tabel basis data oleh ORM Django.
- Migrasi adalah sistem kontrol versi untuk skema basis data Anda, memastikan perubahan pada model diterapkan secara konsisten.
- Django Admin menyediakan antarmuka CRUD yang siap pakai, mempercepat proses pengelolaan konten secara signifikan.

6.5 Tugas

Kerjakan tugas-tugas di bawah ini untuk memperkuat pemahaman dan keterampilan Anda.

1. Apa perbedaan antara perintah `python manage.py makemigrations` dan `python manage.py migrate`? Mengapa keduanya diperlukan?
2. Modifikasi kegiatan praktikum :
 - Modifikasi model `Post` di `blog/models.py`. Tambahkan sebuah bidang (*field*) baru: `category = models.CharField(max_length=100)`.
 - Setelah memodifikasi model, buat dan jalankan migrasi yang diperlukan agar perubahan ini tercermin di basis data dan panel admin.